

# 第五组调研报告

## UF0<sup>2</sup> 架构组件理解与 Linux 安全审计设计思路

项目名称: Linux 桌面 Agentic OS 设计与实现

小组: 第 5 组, 系统协调 + 安全 + RAG

调研主题: UF0<sup>2</sup> 相关组件理解 (Coordinator / Security / Knowledge) 及 Linux 场景下安全与审计设计思路

适用范围: 课程设计调研报告、第五组设计文档补充、系统集成答辩材料

### 摘要

本报告分两部分展开调研。第一部分围绕微软 UF0<sup>2</sup> (UF0<sup>2</sup>: The Desktop AgentOS) 的系统级组件, 重点分析与第五组职责直接相关的三类能力: Coordinator (系统协调与跨模块编排)、Security (安全隔离、权限控制与审计)、Knowledge (持续知识底座与 RAG 轨迹复用)。UF0<sup>2</sup> 的核心价值不只是“让智能体会点鼠标”, 而是把桌面自动化抽象为一种接近操作系统控制面的能力: HostAgent 负责理解用户目标并拆解任务, AppAgent 负责面向具体应用执行, 控制检测模块提供 UIA 与视觉融合的控件定位, Knowledge Substrate 则持续积累应用文档和历史执行经验。

第二部分聚焦 UF0<sup>2</sup> 架构引入 Linux 桌面环境时安全防线的重构思路, 重点分析四类 Linux 原生安全机制: 轻量级无特权沙箱隔离 (Bubblewrap)、内核级细粒度资源限制 (Landlock LSM)、AT-SPI 与 D-Bus 接口的隐私与边界防御, 以及基于工具边界的多维安全审计 (Agent Auditing)。这四类机制与第五组现有四层安全架构 (权限检查 / 沙箱执行检查 / 签名验证 / 隐私保护) 在设计思想上高度吻合, 构成了 Linux Agentic OS 安全体系的完整理论支撑。

根据任务书要求, 本项目需要在 Linux 桌面环境中实现类似 UF0<sup>2</sup> 的 Agentic OS 原型, 因此不能直接复刻 Windows UIA、Win32、WinCOM 和 RDP PiP 等机制, 而应将其思想迁移到 Linux 的 AT-SPI/D-Bus、进程隔离、文件系统权限、

审计日志与轻量级知识检索中。第五组当前代码已经实现 SystemCoordinator、SecuritySandbox、RAGKnowledgeBase、TraceStore 与 AuditLogger 等模块，基本覆盖了 UF0<sup>2</sup> 中“中央编排、隔离执行、长期记忆”的系统性思想。

## 1. UF0<sup>2</sup> 架构背景与第五组定位

UF0<sup>2</sup> 是微软在 UF0 基础上提出的桌面 AgentOS 框架。早期 UF0 主要强调 Windows GUI 场景下的双智能体结构，即 HostAgent 与 AppAgent 协作完成跨应用任务；UF0<sup>2</sup> 则进一步把桌面自动化提升为系统级抽象，引入更深的操作系统集成、更稳定的 GUI + API 混合动作层、持续知识底座、推测式多动作执行，以及隔离的 Picture-in-Picture (PiP) 桌面环境。其目标是解决传统 RPA 和纯视觉 CUA 的共同问题：脚本脆弱、对 UI 变化敏感、缺少应用内部语义、执行时容易抢占用户桌面、缺少长期经验复用。

在 UF0<sup>2</sup> 中，HostAgent 相当于桌面级调度者，负责接收自然语言请求、拆解任务、选择目标应用、协调多个 AppAgent；AppAgent 面向具体应用，负责识别控件、选择 GUI 或 API 动作、执行并反馈结果；Control Detection 通过 Windows UI Automation 与视觉解析融合，解决标准控件和自绘控件的覆盖问题；Knowledge Substrate 则把应用文档、教程、示例轨迹、执行日志等材料转化为可检索知识，在运行时增强智能体提示上下文。微软官方文档将 UF0<sup>2</sup> 概括为深度 OS 集成、混合 GUI/API 动作、持续知识学习和并行非侵入式执行等能力。

本课程项目将 UF0<sup>2</sup> 的 Windows 桌面 AgentOS 思想迁移到 Linux 桌面。第 1 组负责 AI Shell 与 HostAgent，第 2 组负责任务规划和控件检测，第 3 组负责 AppAgent 与 GUI 自动化执行，第 4 组负责工具注册与 OS Skills，第 5 组负责系统协调、安全沙箱与 RAG。因此，第五组不是单纯的“工具调用辅助模块”，而是端到端链路的控制面：在用户意图进入执行系统前进行安全准入，在任务执行过程中统一调度各组模块，在执行后沉淀审计和知识轨迹。

从操作系统原理角度看，第五组对应三个核心主题。第一，虚拟化与资源隔离：安全沙箱通过策略、路径约束、签名验证和隐私保护限制 Agent 对真实系统资

源的访问。第二，并发与调度：协调器通过统一流程、超时控制、重试机制和异步写入，把多个模块组织为可控的执行序列。第三，持久性与存储管理：审计日志和 RAG 知识库把一次性执行转化为可追踪、可检索、可复用的长期记录。这正好对应任务书中“虚拟化、并发、持久性”的课程目标。

## 2. Coordinator：系统协调与跨模块编排

### 2.1 UF0<sup>2</sup> 中的协调思想

UF0<sup>2</sup> 的协调能力主要体现在 HostAgent 和系统运行时中。HostAgent 不是简单的聊天入口，而是桌面任务的总控者：它需要理解用户请求，判断任务涉及哪些应用，决定何时启动或关闭应用，创建和调度 AppAgent，并维护任务执行状态。论文中描述 HostAgent 运行在 Windows 原生底座之上，能够监控活动应用、发出 shell 命令启动进程、管理应用专属 AppAgent 的创建和销毁，并通过状态机控制执行阶段转换。这里的“Coordinator”思想并不局限于一个类名，而是贯穿在任务分解、生命周期管理、跨应用协调、错误恢复和结果汇聚中。

与传统脚本不同，Agentic OS 的协调器面对的是开放桌面环境：应用可能尚未启动，窗口焦点可能变化，UI 元素可能消失，用户可能同时操作系统，LLM 也可能输出不稳定计划。因此协调器必须具备“阶段化执行”的能力。一般可以抽象为：输入校验、意图解析、安全检查、任务规划、控件定位、动作执行、工具调用、结果验证、日志入库、失败恢复。每一阶段都应有明确输入输出和错误边界，避免某一模块异常扩散到整个系统。

UF0<sup>2</sup> 的多智能体分层也体现了操作系统调度思想。HostAgent 负责“做什么、何时做、由谁做”，AppAgent 负责“怎么做、在哪里做”。这种分工类似 OS 中调度器与进程/线程执行上下文的关系：调度器不关心每个应用内部细节，但必须维护全局状态、资源使用、失败处理和任务进度。对课程项目而言，第五组的 SystemCoordinator 就承担了这个控制面角色，只是输入来自第 1 组的结构化 intent\_json，执行能力依赖第 2/3/4 组模块。

### 2.2 本项目 Coordinator 的职责边界

第五组的协调器应当坚持“统一入口、清晰契约、弱耦合调用、强可观测性”四个原则。统一入口意味着所有跨组执行都通过

`SystemCoordinator.orchestrate()` 进入，便于集中做安全、审计和异常处理。清晰契约意味着第 1 组输出 `intent_json`，第 2 组输出 `plan_json`，第 3 组输出 `result_json`，第 4 组输出 `tool_result_json`，第 5 组输出统一 `orchestrate_result`。弱耦合调用意味着协调器只依赖接口能力，不绑定其他组内部实现。强可观测性意味着每次调用都必须携带 `trace_id`，并记录阶段、耗时、风险等级、错误码和结果摘要。

当前代码中的 `coordinator/system_coordinator.py` 已经形成了较完整的编排流程：首先为输入补齐或生成 `trace_id`，然后校验必要字段，再调用 `SecuritySandbox.check()` 进行安全检查；安全通过后，按顺序调用第 2 组规划模块、第 3 组执行模块和第 4 组工具模块；最后将执行结果写入审计日志和 RAG 知识库。该流程与任务书中“第 1 组 → 第 5 组安全检查 → 第 2 组 → 第 3 组 → 第 4 组 → 第 5 组审计/RAG”的链路基本一致。

协调器中值得保留的设计包括：第一，模块注册机制，通过 `register(group_name, module)` 注册外部模块，能够支持 Mock 与真实模块切换，适合跨组并行开发；第二，超时控制，使用 `ThreadPoolExecutor` 对外部模块调用设置 3 秒超时，防止某个组的模块卡死导致整条链路无响应；第三，失败标准化，通过错误码区分安全拒绝、路径穿越、规划不可用、执行不可用、工具不可用和内部异常，便于联调定位；第四，异步入库，RAG 与审计写入不阻塞主流程，降低演示时延。

## 2.3 Coordinator 的 Linux 适配理解

UF0<sup>2</sup> 面向 Windows，可直接利用 UIA、Win32、WinCOM 和 RDP loopback；本项目面向 Linux，需要将协调能力建立在更通用的进程、D-Bus、文件系统和 GUI 自动化工具之上。对于第五组而言，Linux 适配重点不是直接操作 AT-SPI，而是为第 2/3 组提供稳定的调用框架和状态管理：第 2 组可通过 AT-SPI/D-Bus 获取控件树，第 3 组可通过 PyAutoGUI 执行动作，第 5 组需要判断何时调用、如何超时、失败后如何返回、如何记录现场。

协调器还应该为演示场景设计可降级路径。任务书要求当其他组落后时第五组

提供备用 Mock 实现，因此 SystemCoordinator 不应假设所有模块总是存在。当前代码对第 2/3 组未注册会返回标准错误，对第 4 组工具调用则采取可选和降级处理，这种差异是合理的：规划和执行是核心路径，工具调用则可能只在某些任务中出现。

## 3. Security：安全沙箱、权限控制与审计

### 3.1 为什么桌面 AgentOS 必须内建安全层

桌面智能体和普通聊天机器人最大的区别，是它能够直接影响真实操作系统：移动文件、打开网页、执行命令、读取隐私文件、修改系统设置，甚至可能删除数据或泄露凭据。UFO<sup>2</sup> 通过 PiP 虚拟桌面、原生 API 检查、状态机和动作验证来降低误操作风险。论文中提到 PiP 将自动化执行放入隔离的虚拟桌面会话，使 Agent 与用户主桌面并行运行，减少对用户输入设备的抢占，也提供了一种 GUI 自动化隔离原语。虽然 PiP 更偏执行环境与用户体验，但它体现了 Security 的核心原则：Agent 的能力越接近 OS 控制面，就越需要隔离、准入、验证和可追溯。

对 Linux Agentic OS 原型而言，安全层至少要解决四类风险：第一是破坏性操作，例如 `rm -rf /`、格式化磁盘、关机、修改系统目录；第二是越权访问，例如读取 `/etc/passwd`、`/root`、`.ssh/id_rsa` 或用户密钥；第三是命令注入和路径穿越，例如把普通路径包装成 `../../../../etc/passwd`；第四是供应链和接口篡改，例如跨组模块或工具调用参数被伪造。课程项目不要求实现工业级强隔离，但必须展示危险操作被拦截，并能说明安全机制如何与 OS 原理关联。

### 3.2 本项目四层安全架构

任务书要求第五组实现四层安全架构：权限检查、沙箱执行、签名验证、隐私保护。当前 `security/security_sandbox.py` 已经按照这个结构实现，形成了较清晰的层次化防线。

第一层是权限检查。系统通过策略表匹配动作和目标路径，对危险命令、受保护路径、高风险删除等操作进行拦截或风险标记。这相当于操作系统访问控制中的策略决策点，负责判断某个主体是否可以对某个客体执行某类动作。由于

Agent 的主体身份通常来自自然语言意图，策略匹配要同时关注 `action`、`target` 和 `raw_text` 的语义风险。

第二层是沙箱执行检查。当前实现将写类操作限制在 `/home/`、`/tmp/`、`/var/tmp/` 等可控目录中，拒绝向系统关键目录写入。这是课程原型中非常实用的隔离策略：它不依赖复杂容器技术，也能展示“资源隔离”的 OS 思想。

第三层是签名验证。当前实现支持 HMAC-SHA256：当请求参数携带 `signature` 时，必须与 `action|target|params` 生成的签名一致，否则拒绝。这一层可以防止跨组接口传输中参数被篡改，也能为第 4 组工具调用提供来源可信性基础。

第四层是隐私保护。当前实现匹配 `password`、`api_key`、`token`、`secret`、`.ssh`、`id_rsa` 等敏感模式，并对读取、打开、导航等动作进行拦截。这与 AgentOS 的隐私边界密切相关：自然语言代理可能被用户误导，也可能被网页内容间接提示攻击，因此对凭据、密钥和隐私目录应默认更保守。

### 3.3 安全结果与审计闭环

安全检查不能只返回“通过/拒绝”，还应该返回风险等级、拒绝原因和命中策略。当前接口 `CheckJSON` 包含 `approved`、`risk_level`、`reason`、`matched_policy`，符合任务书要求。风险等级可用于协调器决策：低风险直接放行，中风险记录审计，高风险要求用户确认，`critical` 直接拒绝。

审计是 Security 的另一半。没有审计，安全策略只能在运行时拦截，无法事后复盘。第五组的 `AuditLogger` 与 `SystemCoordinator._write_audit()` 将安全通过、拦截、规划完成、执行完成、工具调用、失败等事件写入日志，使每个 `trace_id` 都能回放。这一点非常适合答辩展示：可以先输入危险指令，看到系统返回 E1001 或 E1002，再打开审计日志证明拦截原因和策略编号。

### 3.4 Security 的不足与改进方向

当前安全沙箱已经满足课程验收的核心要求，但仍有若干可说明的局限。第一，策略主要基于字符串和路径规则，对复杂自然语言绕过的理解有限。第二，未真正创建 OS 级隔离环境，文件系统写约束还停留在策略层。第三，高风险但

非绝对危险的操作缺少用户确认机制。改进方向包括：引入操作前预览与确认、使用 Bubblewrap 建立真实沙箱目录、对工具调用实施强签名、对隐私路径采用默认拒绝策略。

## 4. Knowledge：知识底座、执行轨迹与 RAG

### 4.1 UF0<sup>2</sup> Knowledge Substrate 的作用

UF0<sup>2</sup> 的 Knowledge Substrate 是其区别于一次性桌面自动化脚本的重要组件。桌面任务通常具有明显的经验性：同一个应用的菜单位置、按钮名称、快捷键、错误弹窗、文件保存路径、常见失败原因都可以被复用。Knowledge Substrate 通过应用文档、在线教程、API 说明、历史执行轨迹和用户反馈，为智能体提供可检索的上下文，使任务规划和动作选择更接近“带经验的操作员”。任务书中要求第五组实现 RAG 知识库，存储执行轨迹并支持检索，正是对 Knowledge Substrate 的课程化落地。

### 4.2 执行轨迹为什么是 AgentOS 的核心资产

执行轨迹不是普通日志。普通日志主要服务于程序员排错，而 AgentOS 的执行轨迹还要服务于未来任务规划。一次完整轨迹至少包括：用户原始输入、结构化意图、安全检查结果、任务计划、控件检测结果、执行动作、工具调用、执行前后状态、错误信息、耗时、风险等级和最终摘要。轨迹越完整，系统越容易回答三个问题：过去类似任务怎么做成的？这次失败发生在哪个阶段？下次遇到同类任务应该避开什么风险？

例如“整理下载目录”是任务书中的复杂验收场景。若轨迹被保存，第二次用户提出“把下载目录里的图片和文档分类”时，RAG 可以召回上次成功经验，给规划模块提供更稳定的步骤模板；如果上次因为权限不足失败，RAG 也可以提示协调器或安全层提前检查目标目录权限。这种经验复用正是 Knowledge Substrate 的实际价值。

### 4.3 本项目 RAGKnowledgeBase 设计理解

当前代码中的 `knowledge/rag_kb.py` 采用门面模式封装底层 `TFIDFTraceStore`，对外提供 `add_trace()`、`query()`、`get_trace()`、`stats()` 和 `ping()`。协调器在流程完成或失败时调用

`_store_trace_async()` 写入 `TraceRecord`，其中包含 `trace_id`、`action`、`target`、`stage`、`success`、`summary`、`timestamp` 和 `metadata`。底层 `knowledge/trace_store.py` 实现了轻量级 TF-IDF + 余弦相似度检索，并支持 `numpy` 不可用时降级为纯 Python 计算，这对跨平台演示很重要。

`RAGKnowledgeBase.query()` 采用“双通道检索”思想：先由 TF-IDF 召回候选，再通过关键词命中加权重排序。这个设计与 UF0<sup>2</sup> 的 `Knowledge Substrate` 思想一致：知识检索不必追求单一算法，而应结合结构化字段和自然语言摘要，让召回结果对执行有用。后续如果接入 LLM 或 Embedding 模型，可以把当前 `TraceRecord` 作为稳定数据模型保留，只替换底层向量化实现。

## 4.4 Knowledge 与审计、安全、协调的关系

知识库不应孤立存在。它与 `Coordinator` 的关系是“执行后沉淀、执行前辅助”；它与 `Security` 的关系是“风险样本库”：所有被拒绝的危险操作也应写入失败轨迹，形成安全测试样本；它与 `AuditLogger` 的关系是“一个偏检索，一个偏证据”。两者都依赖 `trace_id` 串联。当前实现已经在 `_fail()` 中将失败记录也写入 RAG，这一点很有价值——从 AgentOS 角度看，失败也是知识资产。

# 5. Linux 场景下安全防线重构：Linux 原生机制分析

## 5.1 为什么 Linux 需要重新设计安全防线

UF0<sup>2</sup> 在 Windows 环境中依赖 PiP 虚拟桌面进行 GUI 隔离，依赖 Win32/UIA 权限模型进行访问控制，依赖 WinCOM 接口进行签名可信验证。这些机制与 Windows 内核和桌面会话管理深度耦合，无法直接在 Linux 上复用。Linux 内核和桌面生态为此提供了一套完整的原语：Bubblewrap 提供无特权命名空间沙箱，Landlock 提供进程自我限制的 LSM 机制，AT-SPI 提供控件角色过滤接口，D-Bus 提供消息超时与策略控制，工具边界审计则将所有安全事件转化为可追溯的证据链。

## 5.2 轻量级无特权沙箱隔离（Bubblewrap）

Bubblewrap (bwrap) 是一个轻量级命令行沙箱工具，被 Flatpak 等主流 Linux 应用分发框架广泛采用。它利用 Linux 内核的用户命名空间 (user

namespace)、挂载命名空间 (mount namespace)、PID 命名空间等机制, 在不需要 root 权限的情况下为进程创建隔离的执行环境。其核心安全设计体现在两点: 命名空间隔离使沙箱内进程拥有独立的文件系统视图与 PID 空间; seccomp BPF 过滤规则精确限制允许调用的系统调用集合, 对 execve、socket、ptrace 等高风险系统调用进行拦截。

Bubblewrap 剥夺了沙箱内的 CAP\_SYS\_ADMIN、CAP\_NET\_ADMIN 等能力集, 且强制规定子进程无法获得比父进程更高的操作权限 (no\_new\_privs 标志), 从源头阻断了恶意指令的提权路径。通过只读绑定挂载, 沙箱可只暴露智能体执行任务所必需的目录, 将系统目录 (/etc、/usr、/root、/boot) 完全排除在沙箱视图之外。第五组现有第二层安全检查 (沙箱执行检查) 是合理的策略层隔离, Bubblewrap 代表了在此基础上进一步增强的工程方向。

### 5.3 内核级细粒度资源限制 (Landlock LSM)

Landlock 是 Linux 内核 5.13 版本引入的可堆叠安全模块 (LSM), 其设计目标是赋予无特权进程安全地限制自身访问权限的能力。与传统 LSM (如 SELinux、AppArmor) 需要管理员预先配置全局策略不同, Landlock 允许进程在运行时主动声明自己的最小权限集合, 实现“按需赋权”的最小化授权原则。策略一旦提交即不可放宽, 子进程继承父进程约束并可叠加更严格限制。

将 Landlock 与 Agent 执行器结合, 可以构建“按需赋权”的安全层。在任务开始前, 协调器根据任务类型声明执行器仅需访问的目录集合, 提交 Landlock 策略后启动执行进程, 即使 LLM 生成了越界操作, 执行进程在内核层面也无法完成该操作。Landlock 的思想与第五组第一层权限检查和第二层沙箱执行检查的设计目标高度一致: 两者并不互斥, 用户态检查速度快、可提供精确错误码, 内核层 Landlock 则作为最终兜底防线。

### 5.4 AT-SPI 与 D-Bus 接口的隐私与边界防御

AT-SPI (Assistive Technology Service Provider Interface) 是 Linux 桌面的无障碍功能接口, 允许客户端程序遍历桌面上所有应用的控件树, 读取控件角色、名称、状态和可执行动作。AT-SPI 在设计上面向辅助需求, 不假设调用者存在恶意——这意味着 LLM 驱动的 Agent 可以通过 AT-SPI 读取密码管

理器输入框内容、终端输出文本，甚至触发密码字段的值变更。

自动化操作引擎必须强制屏蔽 `PASSWORD_TEXT` 等敏感角色属性，防止智能体读取密码管理器、终端等高隐私数据。第 2 组控件检测模块应在控件信息返回给协调器之前进行脱敏处理，第五组隐私保护层则在意图层面拦截包含 `password`、`api_key`、`token`、`.ssh`、`id_rsa` 等敏感模式的目标，形成“意图层 + 控件层”的双重防线。为了防止死循环操作导致系统级卡死，必须对所有的 D-Bus 调用强制执行超时限制（Timeouts）。

### 5.5 基于工具边界的多维安全审计（Agent Auditing）

工具函数构成了 LLM 生成内容触及真实系统的执行边界，审计系统必须在这些关键节点进行数据流分析和权限风险排查。先进的 LLM 智能体审计要求将语义层面的检查与在沙箱中观测到的实际状态变更（Sandbox-observed harm）分离记录：语义审计记录记录 LLM 输出的动作描述、安全策略命中结果、风险等级评分；沙箱观测记录记录工具实际执行后的文件系统变更、进程状态变化等可观测物理状态。两类记录通过 `trace_id` 关联，使安全分析人员能够精确定位语义与执行之间的偏差来源。

每次操作执行后，相关带有时间戳的审计日志会由系统协调器捕获，并作为执行轨迹固化存储到 RAG 知识库中，为事后安全溯源提供完整依据。第五组现有实现中，`AuditLogger` 记录全链路审计事件，`RAGKnowledgeBase` 存储执行轨迹，`SystemCoordinator._fail()` 在失败时同时写入审计和 RAG 失败记录，已经体现了审计与知识库闭环的核心思想。

### 5.6 四类机制与第五组实现的整体对照

| Linux 安全机制                  | 对应第五组安全层                | 当前实现状态                   |
|-----------------------------|-------------------------|--------------------------|
| Bubblewrap 命名空间沙箱 / seccomp | 第二层：沙箱执行检查（写目录约束）       | 策略层实现，Bubblewrap 集成为增强方向 |
| Landlock LSM 最小权限           | 第一层：权限检查 + 第二层：路径约束     | 策略层实现，Landlock 集成为增强方向   |
| AT-SPI 角色过滤 / D-Bus 超时      | 第四层：隐私保护（敏感模式拦截）+ 协调器超时 | 意图层拦截 + 调用超时已实现          |
| Agent 工具边界审计                | 审计日志 + RAG 执行轨          | AuditLogger +            |

|  |   |                      |
|--|---|----------------------|
|  | 迹 | RAGKnowledgeBase 已实现 |
|--|---|----------------------|

这一对应关系说明，第五组的安全设计具有 Linux 系统安全理论支撑的分层防御体系，课程原型阶段在用户态实现策略层，为后续在内核机制层面的增强预留了清晰的升级路径。

## 6. 与第五组现有实现的对照分析

从任务书验收点看，第五组需要交付 SystemCoordinator、SecuritySandbox、RAGKnowledgeBase 和审计日志。当前仓库实现与要求基本一致。SystemCoordinator 已实现模块注册、健康检查、输入校验、安全检查、规划调用、执行调用、可选工具调用、异步 RAG 入库、审计写入、统计信息和优雅关闭，对应 UF0<sup>2</sup> 中的系统控制面，是跨组集成的主入口。SecuritySandbox 已实现权限检查、沙箱执行检查、签名验证和隐私保护四层结构，并支持风险等级、策略编号、拒绝原因和风险统计。RAGKnowledgeBase 与 TFIDFTraceStore 已实现执行轨迹存储、相似查询、按 trace\_id 精确查找、统计信息、JSON 持久化和 numpy 降级，对应 UF0<sup>2</sup> 的 Knowledge Substrate。AuditLogger 是 Security 与 Knowledge 的连接层：没有审计，安全策略的效果难以证明；没有轨迹，RAG 无法形成可复用经验。

从工程角度看，现有实现有三个亮点：第一，接口契约集中在 contracts/schemas.py 和 contracts/error\_codes.py，降低了跨组沟通成本；第二，Mock 模块支持独立运行，符合任务书要求的“落后组降级方案”；第三，写入 RAG 和审计均设计为不阻塞主流程，避免知识沉淀拖慢 GUI 演示。

但为了在调研报告和答辩中更完整，也应主动说明改进空间：当前 RAG 仍是轻量级 TF-IDF 检索，不等同于 UF0<sup>2</sup> 可能使用的更复杂知识底座；当前安全沙箱主要是策略层隔离，不是完整虚拟机或容器沙箱；当前协调器采用顺序编排，尚未实现 UF0<sup>2</sup> 提到的推测式多动作并行执行。这些不足不是缺陷，而是课程原型的合理取舍。

## 7. 对课程验收与报告写作的启示

本组调研报告应突出“理解 UFO<sup>2</sup>，但不机械照搬 UFO<sup>2</sup>”。UFO<sup>2</sup> 的原始环境是 Windows，而课程项目目标是 Linux 桌面 Agentic OS；因此，报告中既要说明 UFO<sup>2</sup> 的 HostAgent、AppAgent、Control Detection、Knowledge Substrate 等核心组件，也要强调第五组负责的是系统协调层，并解释为什么 Coordinator / Security / Knowledge 是 AgentOS 能否稳定运行的关键。

在安全机制说明部分，可以结合 Bubblewrap 和 Landlock 的原理，解释为什么第五组采用“策略层隔离 + 路径约束”作为课程原型的工程取舍，并说明在真实系统中如何通过内核机制进一步强化。在隐私防御说明部分，可以结合 AT-SPI 的角色模型，解释 PASSWORD\_TEXT 等敏感角色为什么需要被屏蔽。在审计能力说明部分，可以结合语义审计与沙箱观测分离的思路，展示第五组如何通过 AuditLogger 和 RAGKnowledgeBase 的配合，使每次危险操作拦截都留下可验证的完整证据链。

在验收展示中，第五组可以围绕三个场景组织材料。第一是正常链路：输入“打开文档文件夹”或“整理下载目录”，展示 trace\_id 如何贯穿安全、规划、执行和入库。第二是安全拦截：输入“删除根目录”“访问 /etc/passwd”“读取 id\_rsa”，展示 SecuritySandbox 返回拒绝、风险等级和策略编号。第三是知识检索：执行几条任务后查询“打开文件夹”“下载目录”“创建文件夹”，展示 RAG 能召回相关历史轨迹。

## 8. 结论

UFO<sup>2</sup> 的真正启发在于，它把桌面智能体从单次 GUI 操作提升为系统级运行时：有分层智能体，有结构化控件检测，有混合 GUI/API 动作，有隔离执行环境，也有持续知识底座。第五组负责的 Coordinator / Security / Knowledge 正是 Agentic OS 原型从“能跑”走向“可控、可信、可复用”的关键部分。

在将 UFO<sup>2</sup> 架构引入 Linux 桌面环境时，安全防线的重构必须深度结合 Linux 内核特有的安全机制与无障碍接口特性。Bubblewrap 提供了无特权命名空间沙箱，从执行层阻断提权路径；Landlock LSM 赋予进程自我限制能力，实现“按需赋权”的最小化授权；AT-SPI 角色过滤与 D-Bus 超时控制构建了 GUI 操作

的隐私与稳定边界；基于工具边界的多维审计将语义检查与沙箱观测分离记录，形成可追溯的完整安全证据链。三者结合后，第五组不仅完成了任务书中“系统协调 + 安全 + RAG”的功能要求，也体现了操作系统课程强调的虚拟化、并发和持久性思想。

## 参考文献

- [1] Microsoft Research. UFO<sup>2</sup>: The Desktop AgentOS.  
<https://www.microsoft.com/en-us/research/publication/ufo2-the-desktop-agentos/>
- [2] Microsoft Research. UFO: A UI-Focused Agent for Windows OS Interaction. arXiv:2402.07939. <https://arxiv.org/abs/2402.07939>
- [3] Microsoft. UFO GitHub Repository.  
<https://github.com/microsoft/UFO>
- [4] Microsoft UFO Documentation: UFO2 Overview.  
<https://microsoft.github.io/UFO/ufo2/overview/>
- [5] freedesktop.org. AT-SPI2.  
<https://www.freedesktop.org/wiki/Accessibility/AT-SPI2/>
- [6] Ubuntu Desktop Documentation. AT-SPI Dbus XML interfaces.  
<https://ubuntu.com/desktop/docs/en/latest/reference/accessibility/dbus/>
- [7] PyAutoGUI Documentation. <https://pyautogui.readthedocs.io/>
- [8] Model Context Protocol Specification.  
<https://modelcontextprotocol.io/>
- [9] 操作系统设计任务书: Linux 桌面 Agentic OS 设计与实现
- [10] ArchWiki / Project Github. "Bubblewrap: Low-level unprivileged sandboxing tool used by Flatpak and similar projects."
- [11] The Linux Kernel documentation. "Landlock: unprivileged access control."
- [12] LobeHub Skills Marketplace. "linux-at-spi2: Automation

guidelines for AT-SPI2 DBus XML interfaces.”

[13] arXiv:2603.22853. “Agent Audit: A Security Analysis System for LLM Agent Applications.”

[14] arXiv:2606.18356. “SafeClawBench: Separating Semantic, Audit-Evidence, and Sandbox Harm in Tool-Using LLM Agents.”

[15] Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. Operating Systems: Three Easy Pieces.

[16] Andrew S. Tanenbaum. Modern Operating Systems.